

MASOpt: ONLINE PROMPT OPTIMIZATION FOR MULTI-AGENT SYSTEMS WITH CLOSED-LOOP CONTROL

Anonymous authors

Paper under double-blind review

ABSTRACT

Large language model (LLM)-based multi-agent systems (MAS) decompose a complex task across specialized agents, where the prompt of each agent governs both its own behavior and the information it passes downstream. Existing MAS prompt optimizers search for a configuration before deployment and keep it fixed, which is an open-loop controller that never measures what its own output produces. In this paper, we introduce MASOpt, an online prompt optimization framework that closes this loop with the feedback principle of classical control theory. MASOpt treats the target MAS as a controlled plant and every local prompt edit as a control input whose value is determined only after the plant responds. A trajectory observer estimates a latent failure state from the partial execution, a minimal actuation module turns that state into a local prompt repair, and a feedback gate admits the repair only when the measured error decreases, so the retained prompt state never increases that error. To decide where to actuate instead of where the error becomes visible, we further propose Cross-Actuation Relative Advantage (CARA), which replays candidate actuations from an identical prefix and scores each against the others issued from the same state. Experiments on eight benchmarks show that MASOpt improves the average score from 80.58 to 83.40 over the strongest offline MAS prompt optimizer and raises the average interactive success rate from 39.23 to 64.85, with the backbone LLMs and the workflow topology left unchanged.

1 INTRODUCTION

Large language model (LLM)-based multi-agent systems (MAS) have become a promising paradigm for tasks that exceed the reach of a single model call (Wu et al., 2024; Li et al., 2023; Chen et al., 2023; Zhou et al., 2025). By assigning planning, reasoning, execution, and verification to specialized agents (Li et al., 2023; Chen et al., 2023), a MAS decomposes a hard problem into coordinated sub-processes and aggregates the strengths of many LLM invocations (Wu et al., 2024; Zhang et al., 2025). Within such a system the prompt of each agent specifies its behavior, since it determines how the agent reads its role, consumes upstream messages, and hands information downstream (Khattab et al., 2024; Yuksekgonul et al., 2025b). Optimizing these prompts has therefore become the dominant mechanism for improving a MAS whose backbone models and topology are fixed (Zhang et al., 2026b; Hong et al., 2026; Wang et al., 2026). However, every prompt such an optimizer returns is committed before the task it must solve has started.

Unlike a single-agent prompt, which affects one output, a MAS prompt propagates along the topology, so an edit to an upstream agent changes the input distribution of every downstream agent it feeds (Zhang et al., 2026b; Wang et al., 2026). Prompt quality is therefore a property of the execution state, not of the prompt. An instruction that steers a solver correctly at the start of a derivation turns harmful once the solver emits a malformed expression, because the downstream verifier inherits it and spends the remaining budget confirming it (Hendrycks et al., 2021). The effect is considerably sharper in long-horizon interaction, where one misread dependency propagates through the remaining steps of a 20-step TextCraft episode and a configuration fixed beforehand cannot intervene at the third step even though the failure is already visible (Prasad et al., 2024). In both cases the evidence needed to fix the prompt arrives while the prompt is already in use, and an optimizer that has finished its work cannot read it.

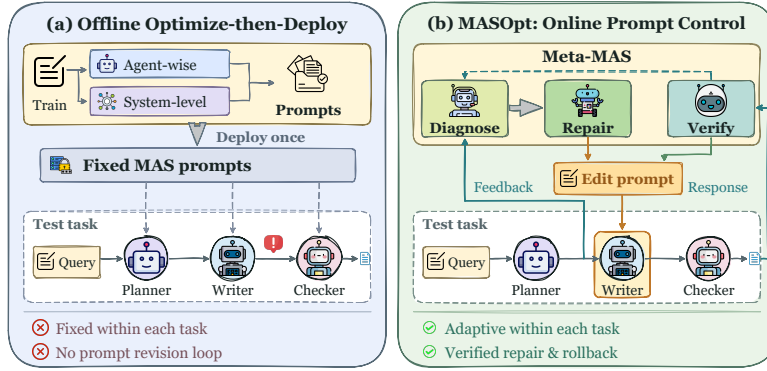


Figure 1: Offline prompt optimization is an open-loop controller that commits a configuration before execution, while MASOpt closes the loop inside the episode and keeps a local prompt repair only when the measured error decreases.

Existing MAS prompt optimization can be broadly categorized into agent-wise and system-level optimization. Early efforts optimize each agent independently with a generic prompt optimizer (Zhou et al., 2023; Pryzant et al., 2023; Yang et al., 2024; Fernando et al., 2024; Lin et al., 2024), treating the remaining agents as a static environment. However, these methods ignore the dependencies the topology induces, so a locally improved prompt can still degrade the workflow containing it (Zhang et al., 2026b). More recently, a line of work models the interaction among prompts explicitly through topology-aware feedback, budget-constrained black-box search, or downstream-utility scoring (Zhang et al., 2026b; Hong et al., 2026; Wang et al., 2026). Although these systems capture the coupling far better, each still returns one configuration frozen at deployment (Zhang et al., 2025; Xia et al., 2026). In essence, both categories are open-loop, as they compute a control sequence in advance and apply it without measuring what it produces.

In this paper, we aim to advance MAS prompt optimization from open loop to closed loop. Classical control theory establishes that an open-loop controller reaches the intended behavior only when the plant model is exact and no disturbance enters the loop, and that feedback is what lets a controller tolerate both (Astrom & Murray, 2008; Goodwin et al., 2001). An LLM-based MAS satisfies neither condition. Its response to a prompt is unpredictable, and every task instance injects a different disturbance. Along with this idea, we propose *MASOpt*, which treats the target MAS as the controlled plant, the observable trajectory as the measurement, and each local prompt edit as a control input. Figure 1 contrasts the two regimes. Specifically, to recover the failure a measurement reveals only partially, MASOpt introduces a *trajectory observer* that estimates a latent failure state. To act on that state without disturbing the workflow, MASOpt restricts the control input to a *minimal actuation* on one agent or one interface, keeping the perturbation small enough for its effect to be attributed. To stop an unverified edit from corrupting the episode, a *feedback gate* admits an actuation only when the measured error decreases and reverts it otherwise. This revert branch is what separates a controller from iterative self-refinement, since Self-Refine rewrites an output and Reflexion accumulates a memory for the next trial, while neither can withdraw a change that made the system worse (Madaan et al., 2023; Shinn et al., 2023).

Closing the loop still leaves open where to actuate. In a MAS the agent at which an error becomes visible is frequently not the agent at which actuation is most effective, since the error is produced upstream and only surfaces downstream (Zhang et al., 2026b). To resolve this, we propose *Cross-Actuation Relative Advantage* (CARA), which replays every candidate actuation from an identical saved prefix and scores it against the others issued from the same state. Fixing the prefix removes the confound of a different preceding trajectory, so the comparison isolates the actuation itself, in the same way that a step response is interpretable only from a known initial condition (Astrom & Murray, 2008; Goodwin et al., 2001). The resulting preferences form a frozen *feedback control memory* that the controller queries as a prior, which lets the loop settle within a few rounds instead of exploring at test time.

Experiments on eight benchmarks covering code generation (Chen et al., 2021; Austin et al., 2021), mathematical reasoning (Cobbe et al., 2021; Hendrycks et al., 2021), reading comprehension (Yang et al., 2018; Dua et al., 2019), and interactive decision making (Shridhar et al., 2021; Prasad et al.,

2024) show that MASOpt improves the average score from 80.58 to 83.40 over MASPOB (Hong et al., 2026) and the average interactive success rate from 39.23 to 64.85. In summary, our key contributions are three-fold.

- **Closed-Loop Formulation.** To the best of our knowledge, MASOpt is the first framework that formulates MAS prompt optimization as closed-loop control, where measurements taken inside an episode determine the prompts governing the remainder of that same episode.
- **Verified Actuation with CARA.** We build a controller from a trajectory observer, a minimal actuation module, and a feedback gate that guarantees a monotone measured error, and propose Cross-Actuation Relative Advantage to learn where to actuate from matched-state replay.
- **Strong Performance.** Experiments on eight benchmarks show that MASOpt outperforms the strongest offline MAS prompt optimizer by 2.82 points on average and by 25.62 points on interactive success.

2 RELATED WORK

Automatic Prompt Optimization The sensitivity of LLMs to natural-language instructions has motivated a broad line of work on automatic prompt optimization. Early approaches select instruction candidates through task-level evaluation (Zhou et al., 2023; Pryzant et al., 2023), OPRO casts optimization as a language modeling problem (Yang et al., 2024), and PromptBreeder evolves the task prompts together with the mutation prompts that modify them (Fernando et al., 2024; Lin et al., 2024). Another group optimizes structured language-model programs, where DSPy compiles prompts as pipeline parameters (Khatab et al., 2024), TextGrad propagates textual feedback in analogy with automatic differentiation (Yuksekgonul et al., 2025b;a), and SkillOpt treats agent skills the same way (Yang et al., 2026). However, all of these optimizers collect their signal at the level of a complete trial, so they learn that a prompt is bad without learning at which point of the trajectory it became bad.

System-Level Prompt Optimization for Multi-Agent Systems Prompt optimization is substantially harder in a MAS, because prompts interact through the topology and improving one agent in isolation changes what every downstream agent receives (Wu et al., 2024; Li et al., 2023). MAPRO casts the problem as maximum a posteriori inference with topology-aware feedback (Hong et al., 2026b), MASPOB as budget-constrained black-box search with a graph neural surrogate (Hong et al., 2026), and MASPO scores a prompt by its contribution to downstream success (Wang et al., 2026). A further line searches the structure of the workflow itself (Zhang et al., 2025; Xia et al., 2026; Zhao et al., 2026). Overall, these methods establish that MAS prompts must be optimized as a coupled system, yet their optimization target remains a configuration selected before deployment.

Runtime Feedback and Online Adaptation A related line studies how language agents improve from feedback at test time. Self-Refine alternates between feedback generation and output refinement (Madaan et al., 2023; Gou et al., 2024), Reflexion converts environmental feedback into verbal reflections kept in an episodic memory (Shinn et al., 2023; Yao et al., 2023), and MemRL learns utilities over such memories while the base model stays fixed (Zhang et al., 2026a). These studies establish the value of non-parametric adaptation, yet their adaptation target is either a single output or a memory consumed by the next episode.

In this work, we focus on the prompt state of a running multi-agent system, with the goal of letting measurements taken inside an episode determine the prompts that govern the remainder of that episode. Unlike previous approaches that treat a configuration as a parameter chosen before deployment, MASOpt adopts a more deliberate design by treating every prompt edit as a control input whose effect is measured from the response of the plant. An ineffective edit is reverted, and an unresolved failure triggers another round.

3 MASOPT

We introduce MASOpt, an online prompt optimization framework that applies the feedback principle of classical control to a fixed multi-agent system. Because a plausible prompt edit may resolve the failure, do nothing, or create a new one downstream, MASOpt measures the response of the plant before deciding whether to keep it. The overview of MASOpt is illustrated in Figure 2, and its formulation, controller, offline calibration, and online loop are introduced as follows.

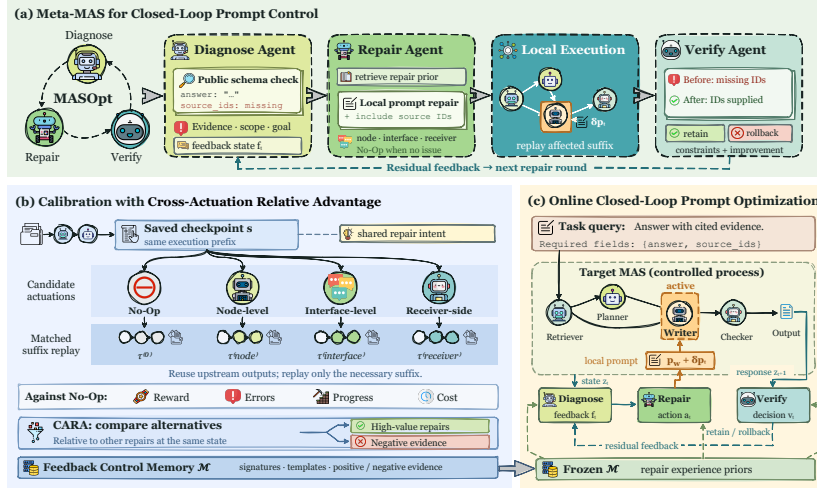


Figure 2: Overview of MASOpt. (a) The controller estimates a latent failure state, actuates a minimal prompt repair, and admits it through a feedback gate. (b) CARA replays candidate actuations from an identical prefix and distills their relative advantage into a feedback control memory. (c) At deployment the memory is frozen and every actuation is retained or reverted by the measured response.

3.1 FROM OPEN-LOOP TO CLOSED-LOOP PROMPT OPTIMIZATION

Plant, Measurement, and Control Input We consider a fixed multi-agent workflow $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node is an agent invocation and each edge carries information from an upstream agent to a downstream one (Zhang et al., 2025). The topology and the backbone LLMs stay fixed (Hong et al., 2026), and the only quantity a controller may change is the prompt state $\mathbf{P}_t = (p_1^t, \dots, p_N^t)$ of the N agents, with $\mathbf{P}_0$ the deployed configuration. For a task x the plant emits a partial trajectory τ_t of responses, inter-agent messages, tool results, and environment observations, of which the controller sees only the measurement $z_t = f_{\text{obs}}(x, \tau_t)$ that excludes hidden labels. A checkpoint is eligible once an agent has produced a complete output that a downstream agent or the environment will consume. At such a checkpoint the controller applies a control input $a_t \in \mathcal{A}(z_t)$, which we call an *actuation*. It edits one agent instruction, one interface constraint, or one receiver-side rule, giving

$$\mathbf{P}_{t+1} = f_{\text{act}}(\mathbf{P}_t, a_t), \quad e_t = \mathcal{E}(z_t), \quad (1)$$

where $f_{\text{act}}(\cdot)$ writes only the local modification a_t specifies, and the *measured error* $\mathcal{E}(\cdot)$ counts the label-free runtime violations the trajectory exposes, including failed public tests, unparseable outputs, missing message fields, unsatisfied dependencies, and invalid environment actions. Counting violations instead of estimating task reward is deliberate, since a violation cannot be traded against correctness. A no-operation input is always available.

Open-Loop and Closed-Loop Objectives An offline optimizer solves the open-loop problem $\max_{\mathbf{P}} \mathbb{E}_{x \sim \mathcal{D}}[R(\tau_T | \mathbf{P})]$ once and applies the solution unchanged, with R the final task metric over the task distribution $\mathcal{D}$ (Hong et al., 2026; Wang et al., 2026). MASOpt instead learns a policy π that emits an actuation at every checkpoint,

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \left[R(\tau_T) - \lambda \sum_{t=1}^T C(a_t) \right], \quad a_t \sim \pi(\cdot | x, z_{\leq t}, \mathcal{M}), \quad (2)$$

where $C(a_t)$ is the cost of an actuation, λ trades control effort against task utility, and $\mathcal{M}$ is the calibrated experience obtained offline. The one difference that matters is that a_t conditions on $z_{\leq t}$, which contains the measurements produced *after* a_{t-1} was applied. The formulation needs only the observable trajectory and a local edit, which lets it control a plant whose response cannot be modeled in advance.

3.2 A CLOSED-LOOP CONTROLLER OVER PROMPT STATE

MASOpt realizes π with three modules that mirror the observer, the actuator, and the feedback path of a regulator (Astrom & Murray, 2008; Goodwin et al., 2001). Separating estimation, actuation, and

acceptance makes an effect attributable, since the module proposing an edit never decides whether it worked.

Trajectory Observer for Latent Failure State A measurement reports the symptom of a failure, so the controller first estimates its cause. The trajectory observer maps the observable history and the retrieved experience to a structured latent failure state

$$s_t = f_{\text{diag}}(x, z_{\leq t}; \mathcal{M}), \quad (3)$$

where s_t records the detected failure, its evidence, its severity, its scope, and the behavior to restore. MASOpt checks the deterministic signals before any semantic estimation, so a failed public test or an invalid environment action enters s_t as hard evidence. The observer answers what must be corrected without committing to an edit, which leaves the actuation space free for the next module.

Minimal Actuation for Local Prompt Repair Large edits are easy to propose and impossible to attribute. MASOpt therefore restricts the control input to the smallest local change that can plausibly remove the diagnosed failure,

$$a_t = f_{\text{rep}}(s_t, \mathbf{P}_t, z_t; \mathcal{M}), \quad (4)$$

where $f_{\text{rep}}(\cdot)$ searches an action space of node-level edits, interface-level constraints, receiver-side rules, and the no-operation input. The retrieved experience supplies a prior over the actuation type, while the current context instantiates the text. MASOpt bounds the perturbation so that the effect of an actuation stays separable from the ordinary variance of the plant, which is what makes verification possible.

Feedback Gate for Verified Acceptance MASOpt treats a proposed repair as a hypothesis, so it replays the affected part of the workflow to a comparable checkpoint and compares the new measurement against the pre-actuation one,

$$g_t = f_{\text{ver}}(s_t, z_t, z_{t+1}; \mathcal{M}), \quad (5)$$

where $g_t \in \{\text{ACCEPT}, \text{REJECT}\}$ carries the decision together with a description of whatever failure remains. The gate admits an actuation only when the measured error decreases and no hard constraint is violated, which yields the closed-loop update

$$\mathbf{P}_{t+1} = \begin{cases} f_{\text{act}}(\mathbf{P}_t, a_t), & \text{if } g_t = \text{ACCEPT and } \mathcal{E}(z_{t+1}) < \mathcal{E}(z_t), \\ \mathbf{P}_t, & \text{otherwise.} \end{cases} \quad (6)$$

The gate is what closes the loop. When an actuation removes only part of the failure, the residual enters the next latent state s_{t+1} and drives another bounded round. This recursion separates MASOpt from one-shot prompt rewriting (Madaan et al., 2023), because the controller never assumes its first actuation was correct.

Property 1 (Monotone measured error). Let $\tilde{e}_t$ denote the measured error evaluated at the prompt state MASOpt holds after the gate at checkpoint t . Under Equation 6 the sequence $\tilde{e}_0, \tilde{e}_1, \dots, \tilde{e}_T$ is non-increasing, so the terminal prompt state satisfies $\tilde{e}_T \leq \tilde{e}_0$.

Neither an open-loop optimizer nor an iterative rewriter has this guarantee. The former never measures its own effect and the latter keeps whatever it produced last, so both can admit an edit that raises the error (Fernando et al., 2024; Madaan et al., 2023). The guarantee holds independently of the observer and of the actuation module, which is the robustness that feedback buys over search (Astrom & Murray, 2008).

3.3 ACTUATION CALIBRATION WITH CROSS-ACTUATION RELATIVE ADVANTAGE

A closed loop still has to choose where to act. With no prior it must find the right actuation by trial at deployment, which is costly and slow to settle. MASOpt therefore calibrates the actuation choice offline, with no LLM parameter updated.

Matched-State Actuation Probing Comparing two actuations applied after different preceding trajectories confounds the effect of the actuation with the effect of the trajectory, so MASOpt replays every candidate from the same saved prefix. Specifically, for an informative checkpoint s we construct a repair intent that states the behavior to restore without prescribing where to apply it, which yields

$\mathcal{A}_s = \{a_0, a_1, \dots, a_K\}$ with a_0 the no-operation input and the remainder legal local actuations at different points of the workflow. MASOpt reuses the unaffected upstream outputs and replays only the necessary suffix, which turns each probe into a controlled response measurement from a known initial condition (Astrom & Murray, 2008; Goodwin et al., 2001).

Cross-Actuation Relative Advantage For an actuation a at state s , we first score the paired outcome against the matched no-operation replay,

$$U(s, a) = \lambda_R \Delta R(s, a) + \lambda_E \Delta E(s, a) + \lambda_P \Delta P(s, a) - \lambda_C C(s, a), \quad (7)$$

where ΔR is the task-reward improvement over the matched no-operation, ΔE the reduction in observable execution error, ΔP the progress toward completion, and C the actuation and replay cost. Absolute utility is not comparable across heterogeneous states, since an easy checkpoint rewards every candidate and a hopeless one rewards none. We therefore score each candidate against the others issued from the same state, which gives the Cross-Actuation Relative Advantage

$$A_{\text{CARA}}(s, a) = \frac{U(s, a) - \mu_{-a}(s)}{\sigma_{-a}(s) + \epsilon}, \quad \mu_{-a}(s) = \frac{1}{|\mathcal{A}_s| - 1} \sum_{b \in \mathcal{A}_s, b \neq a} U(s, b), \quad (8)$$

where $\sigma_{-a}(s)$ is the standard deviation of $U(s, b)$ over the remaining candidates and ϵ stabilizes the ratio. Equation 8 takes the form of a leave-one-out group-relative advantage, with two differences. The group collects actuations applied at *different points of one workflow* instead of repeated samples of one prompt, and the estimate is stored in a memory instead of a parameter update. CARA therefore measures whether an actuation is unusually effective *for this execution state*, which is the question a MAS controller faces, because the node where an error becomes visible is often not the node where actuation has the largest effect (Zhang et al., 2026b).

Feedback Control Memory MASOpt collects the calibrated preferences into a reusable memory $\mathcal{M}$, where each entry records a failure signature, the topology context, an actuation type, a bounded prompt template, and its effectiveness statistics. We keep the actuations of consistently negative advantage as negative evidence, since knowing which repair to avoid at a state removes exploration the online loop would otherwise pay for (Zhang et al., 2026a). At deployment $\mathcal{M}$ is frozen, is queried only as a prior, and never stores task answers.

3.4 ONLINE CLOSED-LOOP OPTIMIZATION

At deployment MASOpt starts every task from $\mathbf{P}_0$ and the frozen $\mathcal{M}$, with no ground-truth answer available. The plant runs normally until an eligible checkpoint, and if the evidence indicates no meaningful failure then no actuation is applied, which keeps the control effort near zero on the tasks the system already solves. Otherwise the observer builds s_t , retrieves matching entries from $\mathcal{M}$, and the actuation module instantiates a minimal edit applied only to the affected part of the workflow, reusing the unaffected upstream computation whenever the environment supports checkpoint replay. The gate then evaluates the post-actuation measurement from runtime signals alone, and Equation 6 retains or reverts the prompt state. The loop stops when the residual is empty, when the gate rejects every candidate, or when the round budget is exhausted. Every actuation is cleared at termination, so MASOpt adapts within a task while the target models, the topology, and the cross-task memory stay fixed, which is the opposite of a memory that accumulates across episodes (Shinn et al., 2023; Zhang et al., 2026a).

4 EXPERIMENTS

We evaluate whether closing the loop inside an episode still improves a multi-agent system that a strong offline optimizer has already tuned.

4.1 EXPERIMENTAL SETTING

Benchmarks We evaluate MASOpt on eight benchmarks covering code generation (Chen et al., 2021; Austin et al., 2021), mathematical reasoning (Cobbe et al., 2021; Hendrycks et al., 2021), reading comprehension (Yang et al., 2018; Dua et al., 2019), and interactive decision making (Shridhar et al., 2021; Prasad et al., 2024). Benchmark details refer to Appendix A.1.

Table 1: Performance (%) on six non-interactive benchmarks. Baselines are ordered by increasing average and MASOpt is separated at the bottom. ‘Avg.’ is the unweighted six-benchmark mean and the best results are marked in **bold**.

Method	HumanEval	MBPP	GSM8K	MATH	HotpotQA	DROP	Avg.
IO	89.31	69.11	87.80	51.71	60.36	53.09	68.56
CoT	89.57	69.89	88.34	52.47	67.62	58.27	71.03
ReAct	87.79	66.08	88.91	52.61	65.61	67.25	71.38
PromptBreeder	88.80	70.38	91.97	52.13	68.76	71.85	73.98
Instinct	90.08	70.23	92.64	52.40	69.92	71.90	74.53
HiveMind	89.31	71.55	92.99	57.41	67.23	68.76	74.54
AFlow	91.09	79.96	93.36	53.83	73.42	79.48	78.52
MIPRO	91.35	80.65	92.80	54.90	74.37	79.13	78.87
MASPOB	94.15	80.65	93.90	57.05	75.43	82.28	80.58
MASOpt	96.95	84.16	94.41	62.76	76.49	85.66	83.40

Baselines We compare MASOpt with reasoning-based prompting (Wei et al., 2022; Yao et al., 2023), prompt optimization (Fernando et al., 2024; Lin et al., 2024; Opsahl-Ong et al., 2024; Hong et al., 2026), and workflow generation (Zhang et al., 2025; Xia et al., 2026). MASPOB is the strongest published system-level MAS prompt optimizer (Hong et al., 2026), and baseline results on the six non-interactive benchmarks follow its evaluation setting. Detailed description of baselines refer to Appendix A.2.

Configuration Unless otherwise stated, every agent of the target MAS is served by `gpt-4o-mini` with temperature 0 and top- p 1, and the backbone parameters stay frozen (Yang et al., 2026). Specifically, MASOpt runs on the same locally executed workflow as the within-system baselines, so the topology, the roles, and P_0 are identical across arms. Ground-truth rewards enter only the offline utility of Equation 7 and are never exposed to the online gate. Interactive runs use a 35-step limit on ALFWorld and a 20-step limit on TextCraft with at most two actuation rounds per checkpoint. We report pass@1 on the code benchmarks, accuracy on the mathematical ones, F1 on the reading benchmarks, and success rate on the interactive ones. Further details refer to Appendix A.3.

4.2 MAIN RESULTS

Capability on Program Synthesis and Mathematical Reasoning We evaluate MASOpt on HumanEval, MBPP, GSM8K, and MATH to assess its *verifiable reasoning capabilities*, where every intermediate failure leaves an executable trace a controller can measure without labels. We compare with PromptBreeder (Fernando et al., 2024), Instinct (Lin et al., 2024), MIPRO (Opsahl-Ong et al., 2024), AFlow (Zhang et al., 2025), and MASPOB (Hong et al., 2026) on the same backbone. As shown in Table 1, MASOpt outperforms MASPOB by 2.80 and 3.51 points on the two code benchmarks and substantially by 5.71 points on MATH. The size of each gain tracks how much room a task leaves for repair, since GSM8K already saturates at 93.90 under the offline optimizer while MATH carries the longest derivations and gains the most. An offline optimizer cannot produce this pattern, because it applies the same instruction to a saturated task and to an unsaturated one, while MASOpt spends an actuation only where a trajectory exposes a failure. Overall, these results highlight that a measurable runtime error signal is what turns a fixed prompt configuration into a correctable one.

Capability on Multi-Hop and Discrete Reasoning We further evaluate MASOpt on HotpotQA and DROP to assess its *evidence-grounded reasoning capabilities*, where a failure is an incomplete aggregation of retrieved evidence instead of a hard execution error. We compare against the same baselines under the identical workflow. As shown in Table 1, MASOpt improves over MASPOB by 1.06 and 3.38 points. Notably, the asymmetry between the two benchmarks is informative, since DROP failures surface as arithmetic and span-selection violations that the observer detects directly, while HotpotQA failures are semantic and leave a particularly weak label-free signal (Yang et al., 2018; Dua et al., 2019). The formulation predicts this dependence, since the strength of the feedback signal bounds how much a controller can repair. Overall, these results highlight that MASOpt converts observable constraint violations into targeted prompt repairs.

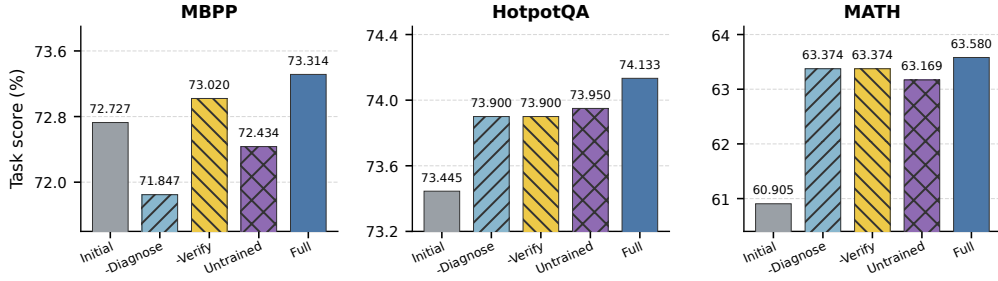


Figure 3: Experience ablations on MBPP, HotpotQA, and MATH. Each panel compares the full controller with the variants that remove diagnostic experience, verification experience, or all learned experience, where every arm reuses the same Initial outputs.

Capability on Long-Horizon Interactive Decision Making We evaluate MASOpt on ALFWorld and TextCraft to assess its *long-horizon recovery capabilities*, where an early mistake compounds over the remaining steps and no single final answer exists. We compare with the same baselines under the shared typed action interface and the identical step limits. As shown in Table 2, MASOpt solves 24 of 80 TextCraft tasks and significantly lifts the three-column average from 39.23 to 64.85. This margin is far larger than on the non-interactive benchmarks, which follows from the formulation, since an interactive episode emits a dense stream of dependency and inventory violations the observer reads at every step while a single-turn task emits one measurement at the end. The unseen split gains more than the seen split for the same reason, because an unfamiliar layout produces more invalid actions for the observer to read. Overall, the benefit of closing the loop grows with the number of measurements the plant makes available.

Table 2: Interactive success rate (%) under the shared action interface and the identical step limits. ‘Avg.’ weights the three score columns equally and the best results are marked in **bold**.

ALFWorld				
Method	Seen	Unseen	TextCraft	Avg.
HiveMind	28.57	27.61	12.50	22.89
MASPOB	47.86	50.75	13.75	37.45
MIPRO	51.43	50.00	12.50	37.98
AFlow	50.71	54.48	12.50	39.23
MASOpt	75.00	89.55	30.00	64.85

5 ANALYSES

Each study below tests a prediction that the formulation makes, asking in turn whether the closed loop depends on its calibrated experience, whether it improves the instructions themselves, and whether it survives a change of workflow architecture and of backbone model.

5.1 EFFECT OF DIAGNOSTIC AND VERIFICATION EXPERIENCE

The feedback control memory is central to MASOpt, as it supplies the prior that keeps the online loop short. To investigate its effect, we conduct an ablation study that removes the calibrated experience from the observer, from the gate, and from both, while the controller and the workflow stay unchanged. As shown in Figure 3, the full controller is the strongest arm on all three benchmarks. In particular, the ordering among the ablations locates the bottleneck of the loop in state estimation. Removing diagnostic experience is the most damaging setting, since an actuation aimed at a misidentified failure cannot reduce the measured error however well the gate is calibrated. Removing verification experience is milder, because the gate retains the deterministic runtime constraints. Overall, the calibrated prior removes the exploration that the loop would otherwise pay for at deployment (Zhang et al., 2026a).

5.2 ADVANTAGE OF THE OPTIMIZED PROMPTS

We further examine whether the closed loop improves the instructions themselves, which task accuracy alone leaves undetermined. We score the retained prompts against the offline-optimized ones under an anonymous six-dimension rubric applied in both presentation orders. As shown in Figure 4, MASOpt is preferred on all four datasets, and the distribution across them is more informative than

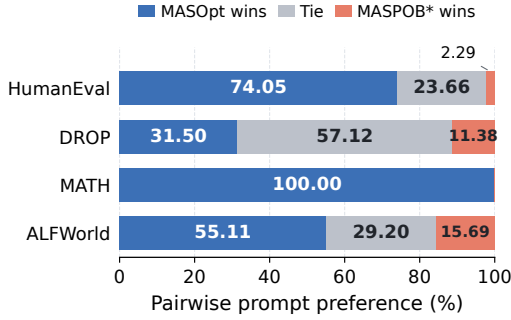


Figure 4: Pairwise prompt preference on four datasets. Segments show MASOpt wins, ties, and baseline wins.

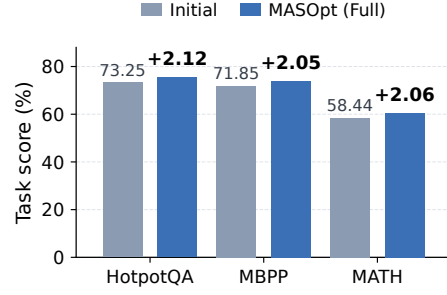


Figure 5: Adaptation to three alternative workflows. Gray bars show Initial and blue bars Full, and labels give the Initial score and the Full-minus-Initial gain in points.

the aggregate, since MATH receives uniform preference while DROP is dominated by ties. This follows from what a closed loop can observe, since a MATH failure yields a derivation defect an actuation can name while a DROP repair leaves the instruction largely intact. Overall, the advantage of MASOpt appears exactly where the runtime signal is specific enough to name the defect.

5.3 GENERALIZABILITY ACROSS WORKFLOW ARCHITECTURES

We then ask how far MASOpt depends on the workflow it repairs. Holding the backbone and the task set fixed, we rerun the identical controller on three simplified architectures. As shown in Figure 5, the closed loop improves every architecture by about two points (Zhang et al., 2025). Overall, a controller that had learned the idiosyncrasies of the original graph would produce margins varying with the branch structure, while one acting on measured error is indifferent to it.

5.4 GENERALIZABILITY ACROSS BACKBONE MODELS

We finally measure how the closed loop behaves under different backbones. We instantiate the identical controller on six models of different families and scales, and pair every run with its own initial workflow. As shown in Figure 6, the paired comparison is non-negative on every model and benchmark. Notably, the single tie occurs where the initial workflow already solves the benchmark completely and leaves no failure to observe. The gate predicts this shape, since it accepts only an actuation that reduces the measured error, so a stronger backbone offers fewer opportunities to act. Overall, MASOpt transfers across model families without structural recalibration, while the size of its benefit follows how much observable failure the backbone leaves behind.

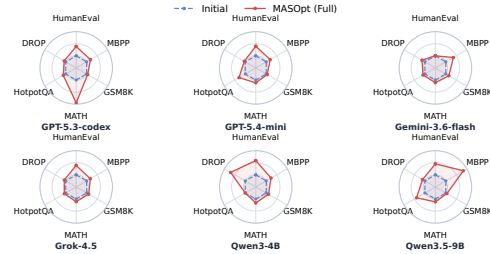


Figure 6: Adaptation across six backbones. Each Initial score is normalized to 100 and all panels share the same radial range.

6 CONCLUSION

We introduce MASOpt, which turns MAS prompt optimization from an open-loop search into a closed-loop control problem, where a trajectory observer estimates a latent failure state, a minimal actuation repairs one prompt, and a feedback gate admits the repair only when the measured error decreases. Experiments on eight benchmarks show that MASOpt improves over the strongest offline MAS prompt optimizer on every benchmark with the backbone models and the workflow topology left fixed.

ETHICS STATEMENT

This work uses publicly available benchmarks and does not involve human subjects, private data, or interventions in real-world decision-making. MASOpt modifies only natural-language prompts and never updates model parameters, so its failure modes stay bounded by the behavior of the underlying models.

REPRODUCIBILITY STATEMENT

The formulation and the controller are specified in Section 3 and the evaluation protocol in Section 4 and Appendix A. We will release the prompt templates, the calibration trajectories, the evaluation scripts, and the configuration files needed to reproduce the reported results, subject to the terms of the underlying model and benchmark licenses.

REFERENCES

- Karl J. Astrom and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. URL <https://fbsbook.org>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Weize Chen, Yushi Su, Jing Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Huan Chan, Yujia Qin, Yaxi Lu, and Bing Xie. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. *arXiv preprint arXiv:2308.10848*, 2023.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 2368–2378, 2019.
- Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 13481–13544. PMLR, 2024. URL <https://proceedings.mlr.press/v235/fernando24a.html>.
- Graham C. Goodwin, Stefan F. Graebe, and Mario E. Salgado. *Control System Design*. Prentice Hall, 2001.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Min Yang, Nan Duan, and Weizhu Chen. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024.

- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- Zhi Hong, Qian Zhang, Jiahang Sun, Zhiwei Shang, Mingze Kong, Xiangyi Wang, Yao Shu, and Zhongxiang Dai. Maspob: Bandit-based prompt optimization for multi-agent systems with graph neural networks. *arXiv preprint arXiv:2603.02630*, 2026.
- Omar Khattab, Keshav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Kevin Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, and Hanna Moazam. Dspy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Xiaoqiang Lin, Zhaoxuan Wu, Zhongxiang Dai, Wenyang Hu, Yao Shu, See-Kiong Ng, Patrick Jaillet, and Bryan Kian Hsiang Low. Use your INSTINCT: INSTRUCTION optimization for LLMs using neural bandits coupled with transformers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 30317–30345. PMLR, 2024. URL <https://proceedings.mlr.press/v235/lin24r.html>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, and Yiming Yang. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Krista Opsahl-Ong, Arnav Singhvi, Harshit Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. *arXiv preprint arXiv:2406.11695*, 2024.
- Prasad et al. TextCraft: A text-based environment for planning and crafting, 2024.
- Reid Pryzant, Dan Iter, Jerry Li, Ying Lee, Chenguang Jung, and Pieter Abbeel. Automatic prompt optimization with “gradient descent” and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7957–7968. Association for Computational Linguistics, 2023. URL <https://aclanthology.org/2023.emnlp-main.494/>.
- Noah Shinn, Federico Cassano, Ashon Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 8634–8652, 2023.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.
- Zhexuan Wang, Xuebo Liu, Li Wang, Zifei Shan, Yutong Wang, Zhenxi Song, and Min Zhang. Maspo: Joint prompt optimization for llm-based multi-agent systems. *arXiv preprint arXiv:2605.06623*, 2026.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 24824–24837, 2022.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.

- Yihan Xia, Taotao Wang, Shengli Zhang, Zhangyuhua Weng, Bin Cao, and Soung Chang Liew. HiveMind: Contribution-guided online prompt optimization of LLM multi-agent systems. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(35):29767–29774, 2026. doi: 10.1609/aaai.v40i35.40222. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40222>.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=BbxyP3xB8A>.
- Yifan Yang, Ziyang Gong, WeiQuan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. Skillopt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2369–2380, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Mert Yuksekogun, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639:609–616, 2025a. doi: 10.1038/s41586-025-08661-4.
- Mert Yuksekogun, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639:609–616, 2025b. doi: 10.1038/s41586-025-08661-4.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bing Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations*, 2025.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, and Yutao Qi. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026a.
- Zheyuan Zhang, Lin Ge, Hongjiang Li, Weicheng Zhu, Chuxu Zhang, and Yanfang Ye. Mapro: Recasting multi-agent prompt optimization as maximum a posteriori inference. In *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 4458–4480. Association for Computational Linguistics, 2026b. doi: 10.18653/v1/2026.findings-eacl.233.
- Mingming Zhao, Xiaokang Wei, Yuanqi Shao, Kaiwen Zhou, Lin Yang, Siwei Rao, Junhui Zhan, and Zhitang Chen. A²Flow: Automating agentic workflow generation via self-adaptive abstraction operators. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(35):29930–29938, 2026. doi: 10.1609/aaai.v40i35.40240. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40240>.
- Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan O. Arik. Multi-agent design: Optimizing agents with better prompts and topologies. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=uCKvHwehlq>.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=92k3aOeLke>.

A EXPERIMENTAL DETAILS

A.1 BENCHMARKS

HumanEval (Chen et al., 2021) and **MBPP** (Austin et al., 2021) evaluate program synthesis from a natural-language specification, where a prediction is correct when it passes the hidden tests. **GSM8K** (Cobbe et al., 2021) and **MATH** (Hendrycks et al., 2021) evaluate multi-step mathematical reasoning, where we adopt the fixed Level-5 test subset of Zhang et al. (2025) that contains 155 Prealgebra, 124 Number Theory, 108 Precalculus, and 99 Counting and Probability problems. **HotpotQA** (Yang et al., 2018) evaluates multi-hop reasoning over two supporting paragraphs, and **DROP** (Dua et al., 2019) evaluates discrete reasoning that requires counting, sorting, and arithmetic over a passage. **ALFWorld** (Shridhar et al., 2021) evaluates embodied household tasks in a text interface with separate seen and unseen splits, and **TextCraft** (Prasad et al., 2024) evaluates compositional crafting in which a target item is obtained only after its recipe dependencies are satisfied in order. Success on the two interactive benchmarks therefore depends on recovering from environment feedback instead of on a single final answer, which is what makes them the setting where a closed loop has the most measurements to act on.

Table 3 lists the calibration pools, test sizes, and evaluation metrics of the eight benchmarks.

Table 3: Datasets, splits, and evaluation metrics. All reported scores are on a 0–100 scale.

Task family	Benchmark	Calibration	Test	Metric
Code generation	HumanEval	33	131	pass@1
	MBPP	86	341	pass@1
Mathematical reasoning	GSM8K	264	1,055	answer accuracy
	MATH	119	486	answer accuracy
Reading comprehension	HotpotQA	200	800	token F1
	DROP	200	800	answer F1
Interactive environments	ALFWorld	33	140 seen / 134 unseen	success rate
	TextCraft	20	80	success rate

A.2 BASELINES

We compare MASOpt with three groups of methods. The first group is direct and reasoning-based prompting, covering **IO** that answers in one call, **CoT** (Wei et al., 2022) that elicits an intermediate reasoning chain, and **ReAct** (Yao et al., 2023) that interleaves reasoning with tool or environment actions. The second group is prompt optimization, covering **PromptBreeder** (Fernando et al., 2024) that evolves task and mutation prompts, **Instinct** (Lin et al., 2024) that searches instructions with a neural bandit, **MIPRO** (Opsahl-Ong et al., 2024) that jointly optimizes instructions and demonstrations of a language-model program, and **MASPOB** (Hong et al., 2026) that treats fixed-topology MAS prompt optimization as budget-constrained black-box search. The third group is workflow generation and adaptation, covering **AFlow** (Zhang et al., 2025) that searches code-represented workflows with Monte Carlo tree search and **HiveMind** (Xia et al., 2026) that attributes credit across agents and refines the lowest-contribution node. Baseline results on the six non-interactive benchmarks follow the evaluation setting of Hong et al. (2026), and the interactive comparison uses the shared typed action interface described below.

A.3 IMPLEMENTATION PROTOCOLS

Every agent of the target MAS is served by `gpt-4o-mini` with temperature 0 and top- p 1, and MASOpt never updates a backbone parameter. The calibration pools listed in Table 3 are divided into three disjoint parts for experience construction, experience selection, and controller selection. Ground-truth rewards enter only the offline utility of Equation 7 and are never visible to the online feedback gate, which decides using the label-free measured error of Equation 1 together with the deterministic runtime constraints.

On the interactive benchmarks, all compared systems use the same typed action interface, the same native environment success condition, and the same step limits of 35 on ALFWorld and 20 on

TextCraft. MASOpt additionally reads the public dependency and inventory constraints that the environment exposes at every step and turns a detected violation into a bounded actuation. Its frozen controller generates task-local repairs from these public observations alone, and it neither updates the cross-task memory nor selects prompts using test accuracy.

A.4 MEASURED ERROR

The measured error $\mathcal{E}$ of Equation 1 counts constraint violations that the runtime exposes without any label. On the code benchmarks it counts syntax errors, raised exceptions, and failures of the public tests attached to the problem statement. On the mathematical benchmarks it counts unparsable final answers, expressions that fail a symbolic well-formedness check, and disagreement among the solver branches of the workflow. On the reading benchmarks it counts answers whose span is absent from the retrieved passages, answers of the wrong type for the question, and missing fields in the message that the aggregator receives. On the interactive benchmarks it counts invalid actions rejected by the environment, actions whose preconditions are unsatisfied, and repeated states that make no progress.

Every term of $\mathcal{E}$ is a violation of a stated constraint and not an estimate of task reward, so reducing it cannot be traded against correctness the way a learned proxy can. The hidden evaluation label never enters $\mathcal{E}$, so the feedback gate of Equation 6 makes its accept and reject decisions from information that is also available at deployment.

A.5 ANALYSIS PROTOCOLS

Prompt quality We anonymize the prompts MASOpt retains at termination together with those the offline optimizer commits before deployment (Hong et al., 2026), then score both with a fixed six-dimension rubric in the two presentation orders and count a difference above 0.5 as a win. The six dimensions cover clarity, specificity, coverage of the task constraints, handling of edge cases, compatibility with the downstream agent, and overall quality.

Alternative architectures We replace the original graphs with one answer-generation call plus formatting on HotpotQA, one code-generation branch plus test and repair on MBPP, and two text solvers with self-consistency on MATH, then run the identical controller on each.

A.6 CONTROL EFFORT

We also record what the closed loop costs. We measure the total API calls of a complete TextCraft test under the shared interface and the identical 20-step limit, then compare them with the two strongest baselines. As reported in Table 4, MASOpt uses 888 calls against 699 and 684, an increase of 27%, and converts it into 24 solved tasks against 10 and 11. The asymmetry between the two ratios follows from Equation 2, since no actuation is applied at a checkpoint whose measurement indicates no failure, which concentrates the budget on the episodes that would otherwise fail. Overall, the control effort scales with the number of detected failures and not with the size of the test set.

Table 4: Control effort and task utility on TextCraft under the shared action interface and the 20-step limit.

Method	Calls	Solved	Succ.
MIPRO	699	10 / 80	12.50
MASPOB	684	11 / 80	13.75
MASOpt	888	24 / 80	30.00

A.7 CONTROLLER HYPERPARAMETERS

The utility weights of Equation 7 are set so that the task-reward term dominates, with $\lambda_R = 1.0$, $\lambda_E = 0.5$, $\lambda_P = 0.3$, and $\lambda_C = 0.1$, and with $\epsilon = 10^{-6}$ in Equation 8. The candidate set at a probed checkpoint contains the no-operation input and $K = 4$ legal local actuations. At deployment the controller retrieves the five memory entries whose failure signature best matches s_t and allows at most two actuation rounds per checkpoint.

A.8 ACTUATION SPACE

The actuation space contains four types of local edit. A node-level actuation rewrites a bounded portion of the instruction of a single agent. An interface-level actuation adds a constraint on the message one agent sends to a specific downstream neighbor. A receiver-side actuation adds a rule governing how an agent consumes a message it receives. The no-operation actuation leaves the prompt state unchanged and is always available, which lets the controller decide that no repair is warranted.

B CASE STUDY